

Performance Intelligence Copilot



IA aplicada para automatizar el análisis y acelerar decisiones sobre pruebas de rendimiento

Grupo 8

Luis Araya
Rodrigo González
Hernán Medina

Profesor guía: Ahmad Armoush · Agosto 2026

01 · CONTEXTO Y EVOLUCIÓN DEL PROYECTO

Del negocio bancario a una decisión de rendimiento explicable

NEGOCIO BANCARIO

Personas y empresas

Acceden a productos y servicios financieros a través de canales que deben responder de forma continua y estable.

WEB

Canales digitales

APP

Operaciones

API

Servicios integrados

CAPACIDAD HABILITANTE

Pruebas de rendimiento

Permiten medir tiempos de respuesta, errores y capacidad antes de liberar cambios.

CAMBIO DE ENFOQUE

PRIMERA PRESENTACIÓN

Predicción de capacidad e infraestructura cloud

El planteamiento era más amplio que los datos y capacidades demostrables del proyecto.



PROYECTO REAL

Copilot para analizar pruebas Gatling

Integra configuración, resultados e historial para entregar una recomendación explicable.

EVIDENCIA REAL

Configuración → métricas → análisis → recomendación → validación humana

Así llegamos al problema: la prueba puede ejecutarse, pero preparar la evidencia, interpretarla y acordar la siguiente acción sigue requiriendo tiempo y criterio especializado.

El cuello de botella no es ejecutar la prueba: es preparar, interpretar y aprobar

3–48 h

preparación y análisis por prueba

El cuello de botella está en consolidar evidencia e interpretar resultados antes de aprobar la siguiente acción técnica.

1–35 h

revisión y aprobación QA

12/mes

volumen de pruebas del equipo

ATENCIONES / AÑO	COSTO ANUAL	COSTO PROMEDIO	INDICADOR RESERVADO
7.003	\$1.569 MM	≈ \$224 mil	—
casos atendidos en el período informado	monto anual informado en la presentación	por atención, según la estimación referida	% incidentes post-falla medido sobre pruebas con fallo detectado

Acelerar la decisión técnica: de horas a minutos

OBJETIVO GENERAL

Integrar configuración, resultados e historial Gatling para recomendar la siguiente acción técnica con trazabilidad y validación humana.

01

Pipeline confiable

Ingesta, validación y normalización reproducible.

02

Motor explicable

Recomendación maintain/review/evolve con reglas y modelo interpretable.

03

Evaluación controlada

Comparación con baseline y control explícito de sobreajuste.

KPI	META	RESULTADO (OPERATIONAL_CORE)	ESTADO	Cobertura	>= 95 % 96,6 % (28/29) Cumple	Macro-F1 operacional	>= 0,55 0,5987 Cumple	Balanced accuracy operacional	>= 0,55 0,62 Cumple
Desv. Macro-F1 operacional	<= 0,20	0,1738	Cumple	Mejora Macro-F1 vs baseline	>= 0,15 +0,1820 pp Cumple				

Las variantes cercanas a las reglas (all_features, without_assertions) alcanzaron 1,00, pero operational_core se usa como resultado principal por excluir proxies cercanos a la etiqueta.

04 · METODOLOGÍA Y ARQUITECTURA

Un flujo híbrido desde la evidencia hasta la recomendación



Composición y calidad del dataset validado

La muestra útil es pequeña: 28 registros. El CSV no trae microservicio explícito, por lo que la procedencia se reconstruye desde el árbol de orígenes de ejemplo.

EMBUDO

29 → 28

1 ejecución fue omitida por duplicado.

CLASE

20 / 8

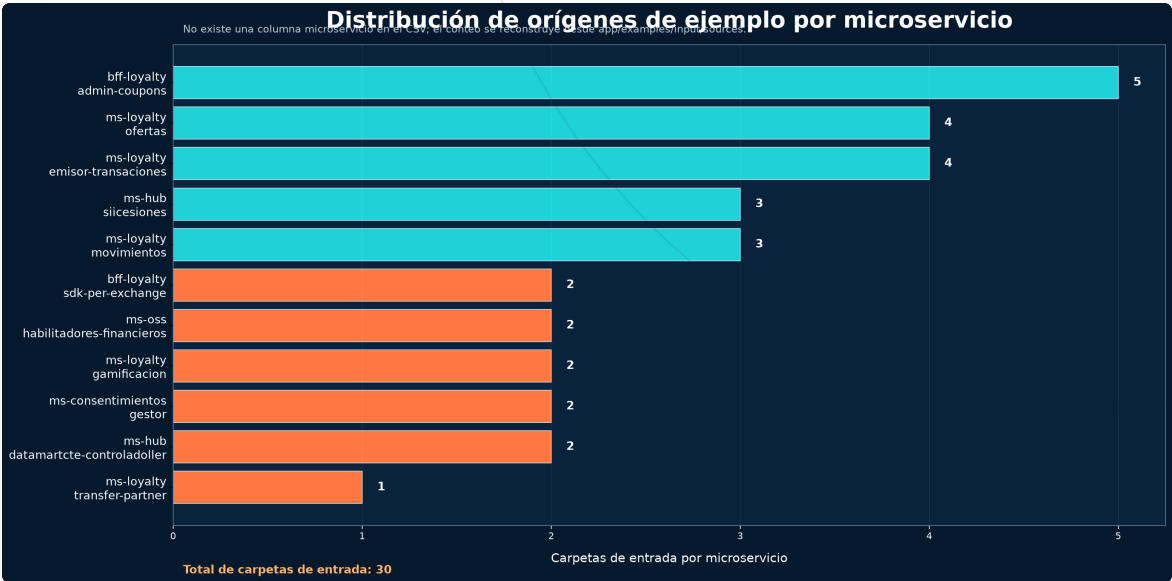
La muestra está desbalanceada.

NULOS

1 columna

p90_response_time_ms quedó vacía.

VARIABLE	PREGUNTA QUE RESPONDE	ROL
error_rate_percent	Señal de falla operacional	Señal
p95_response_time_ms	Latencia del 95 % de solicitudes	Señal
sla_margin_ms	Brecha entre p95 y SLA	Derivada
assertions_failed	Reglas técnicas incumplidas	Señal
warning_count	Advertencias del análisis	Proxy
recommendation_action	Salida: maintain, review o evolve	Etiqueta

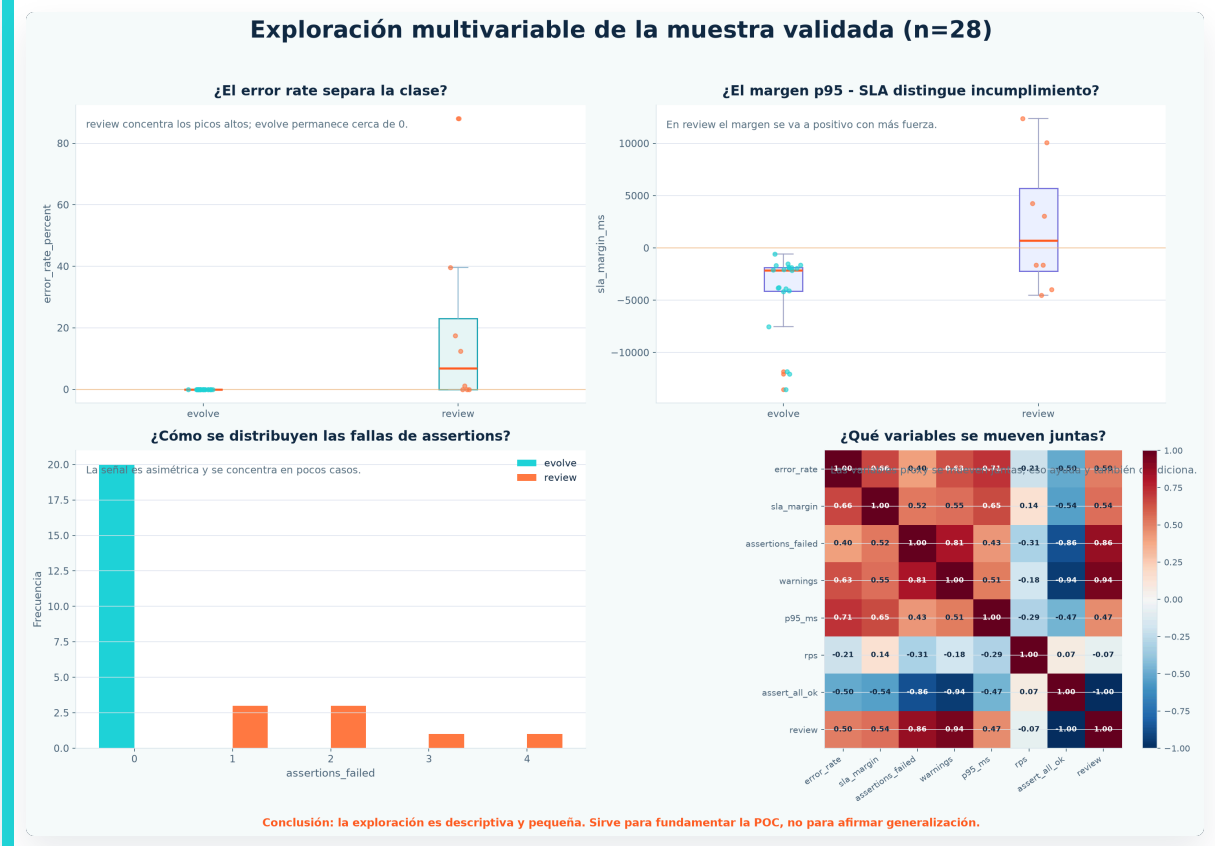


Fuente: árbol app/examples/input/sources. Se usan 30 carpetas de entrada repartidas en 11 microservicios; el CSV final sigue siendo de 28 registros.

Límite estadístico: con 28 registros no se demuestra generalización. Esta slide solo acredita composición, limpieza y procedencia de la muestra.

Las variables separan la clase y muestran proxies

Cada visual responde una pregunta distinta. Con n=28, la exploración es descriptiva y sirve para fundamentar la POC, no para probar un universo distinto.



03 · ASSERTIONS FAILED

las fallas se concentran en pocos casos

La señal es asimétrica y confirma que las reglas pesan, aunque aparecen poco.

04 · CORRELACIONES

hay coherencia interna, no causalidad

Las variables proxy se mueven juntas; eso ayuda al modelo y también lo condiciona.

Lectura metodológica: estas señales sí ayudan a distinguir clases dentro de la muestra. Lo que todavía no prueban es que train y test representen universos distintos ni que las variables funcionen fuera de las reglas que generaron las etiquetas.

ETL e ingeniería de atributos para construir variables auditables

EMBUDO

29 → 28

Se omitió 1 ejecución duplicada; la muestra útil queda en 28 registros.

DISTRIBUCIÓN

20 / 8

El corte final está desbalanceado, por eso se evalúa de forma estratificada.

NULOS

1 columna

p90_response_time_ms quedó vacía en todas las filas y se excluyó del entrenamiento.

CONTROL	QUÉ SE HIZO	POR QUÉ IMPORTA
Outliers	No se aplicó un detector automático. Los extremos se revisaron en la exploración y se conservaron porque son ejecuciones reales y el dataset es pequeño.	Evita perder señal operativa y mantiene representatividad.
Fuga de información	Se excluyen <code>schema_version</code> , <code>metrics_scope</code> y la etiqueta <code>recommendation_action</code> . En <code>operational_core</code> también se quitan <code>assertions_*</code> y <code>warning_count</code> .	Impide que el modelo vea metadatos o proxies demasiado cercanos a la decisión.
Historial comparable	Es la misma estructura de columnas, el mismo esquema y el mismo pipeline de preparación para comparar ejecuciones sin cambiar la regla de lectura.	Permite comparar corridas con una base homogénea y trazable.

PREPROCESAMIENTO REAL ANTES DE ENTRENAR

01

Filtrar columnas válidas

Se usan solo las variables no metadata y no etiqueta. Luego se elimina la columna vacía p90_response_time_ms.

02

Imputar faltantes

Las numéricas usan mediana; las categóricas usan el valor `__missing__`.

03

Codificar categóricas

load_type y assertions_all_passed pasan por OneHotEncoder. No se usa escalamiento.

04

Entrenar el árbol

El modelo final es DecisionTreeClassifier con max_depth=4, class_weight="balanced" y random_state=42.

Margen SLA: `sla_margin_ms = p95_response_time_ms - strictest_response_time_target_ms`. Si el resultado es positivo, el p95 supera el SLA; si es negativo, cumple.

Límite estadístico: la detección de extremos es exploratoria, no algorítmica. Con 28 registros no se prueba generalización; solo se documenta cómo se limpió y transformó la muestra antes de entrenar.

Qué aporta hoy el árbol y qué deberá aprender después

Evidencia	Qué permite afirmar	Qué no permite afirmar
Variantes cercanas a reglas	all_features y without_assertions llegan a Macro-F1 = 1,00 y balanced accuracy = 1,00 en 30 particiones	Eso sí reproduce etiquetas
Señales operacionales	operational_core baja a Macro-F1 = 0,5987 y balanced accuracy = 0,62	La señal operativa sola es más débil
Lectura metodológica	Las variables cercanas al motor de reglas reproducen las etiquetas; al restringirlas a señales operacionales, el desempeño cae	No debe leerse como reproducción fuerte de reglas expertas

APORTE DEL MACHINE LEARNING · CONVERGENCIA CON REGLAS EXPERTAS, NO SUSTITUCIÓN

IMPORTANCIA APRENDIDA POR EL ÁRBOL · operational_core (sin proxies)

VARIABLE	IMPORTANCIA
ROL OPERACIONAL	
error_rate_percent	Porcentaje de solicitudes fallidas en la ejecución
min_response_time_ms	Tiempo mínimo de respuesta; señal de degradación base

Fuente: historical_model_explanation.json · importancia Gini del árbol entrenado sobre el perfil operational_core (18 variables, sin assertions ni warning_count).

HALLAZGO DE ESTA MUESTRA

Sin proxies de reglas, el árbol se apoya en error_rate_percent y min_response_time_ms con igual importancia.

Esto sugiere que la tasa de error y la latencia mínima son las señales operativas más informativas disponibles para separar las clases en esta muestra.

SIGUIENTE VALIDACIÓN

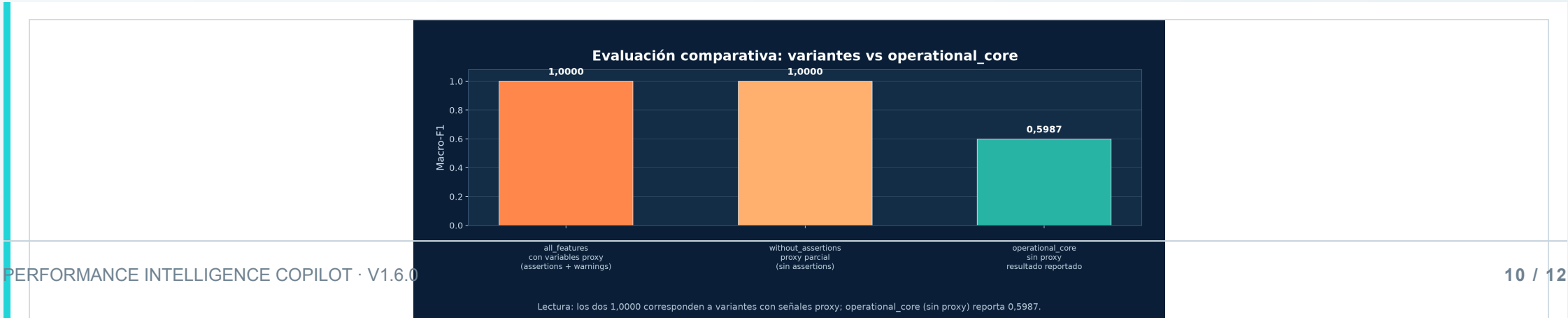
Desacoplar etiquetas y señales operacionales

Separar entrenamiento/prueba por microservicio y usar etiquetas expertas independientes para medir generalización real.

Alcance: el experimento no demuestra reproducción fuerte de reglas expertas. Demuestra que las variables cercanas al motor de reglas elevan el desempeño, mientras las señales operacionales reducen el Macro-F1 y la balanced accuracy.

Las variantes cercanas a reglas superan a las operacionales

CONCLUSIÓN DEL EXPERIMENTO La comparación separa señales cercanas a reglas de señales operacionales Las variantes con variables cercanas al motor de reglas alcanzan 1,00; al restringirlas a señales operacionales, el desempeño desciende a Macro-F1 0,5987 y balanced accuracy 0,62.	DATOS EVALUADOS 28 ejecuciones 20 evolve · 8 review	PROTOCOLO 30 particiones holdout estratificado · 21/7 · random_state 42
	OPERATIONAL_CORE Macro-F1 0,5987 balanced accuracy 0,62 · review F1 0,3968 · std 0,1738	BASELINE MAYORITARIO Macro-F1 0,4167 accuracy 0,7143 · balanced acc. 0,50 · review F1 0,00



10 · IMPACTO OPERACIONAL ESPERADO

Impacto operacional y evaluación económica preliminar

ESCENARIO A · ACTUAL

3–48 h + 1–35 h

Desarrollo de la prueba y posterior revisión QA. El tiempo específico de preparación analítica se medirá en el piloto.



ESCENARIO B · CON COPILOT

Minutos + validación humana

La preparación analítica y la recomendación se automatizan; QA recibe evidencia explicable para revisar y aprobar con mayor rapidez.

BASE HISTÓRICA

\$1.569 MM ÷ 7.003

≈ \$224 mil por atención

ALCANCE INICIAL

144 atenciones/año

12 atenciones mensuales × 12 meses · ≈ \$32,3 MM

ESCENARIO PRELIMINAR

50 % cobertura × 75 % reducción

Supuestos de adopción y eficiencia · ≈ \$12,1 MM potencial/año

Supuestos: 144 atenciones = 12 mensuales × 12 meses; 50 % es cobertura inicial estimada y 75 % es una meta preliminar de reducción del esfuerzo abordado. Deben validarse en un piloto antes de considerar ahorro efectivo.

Una base explicable para decisiones de rendimiento más consistentes

CONTRIBUCIÓN

La POC transforma resultados Gatling en datos estructurados y demuestra que un árbol interpretable puede aprender las etiquetas del motor experto cuando utiliza variables cercanas a esas reglas. Al restringirlo a señales operacionales, **supera el baseline**, aunque su generalización aún debe validarse con nuevos datos y etiquetas independientes.

Siguiente validación

- Validar las recomendaciones con especialistas
- Ampliar el histórico con nuevos casos etiquetados
- Medir tiempo de análisis y reejecuciones en un piloto

La generalización deberá validarse con nuevos datos y revisión experta.